

NovaOS: A Bare-Metal x86_64 Kernel with a Companion Simulated-Desktop Environment

CIT 322 :: Operating System Sessional

Name: **Rajesh Biswas**

ID: **2002060**

Reg: **09577**

Semester: **6th**

1. Introduction

This project demonstrates core Operating System principles through the ground-up development of **NovaOS**, which has two complementary parts. The first is a **bare-metal kernel** for the x86_64 architecture, written in C and booted by the modern **Limine** bootloader. It boots straight into 64-bit long mode, brings up a linear-framebuffer text console and a 16550 UART serial debug log, and drops into an interactive command shell driven by a polled PS/2 keyboard. The second is **NovaOS Desktop**, a companion application written in Python and PyQt5 that simulates an operating-system desktop environment — wallpaper, taskbar, start menu and draggable windows — populated with a suite of built-in applications.

The kernel is compiled with **clang** acting as a cross-compiler (so no GCC cross-toolchain has to be built), linked with GNU ld using a higher-half link script, and mastered into a hybrid BIOS/UEFI ISO that runs under QEMU. Every subsystem is a small, self-contained module so the code stays readable and easy to maintain. Together the two parts illustrate OS concepts from the boot handoff and device drivers up to a usable graphical environment.

2. Objectives

1. To understand the x86_64 boot process and CPU bring-up by building a Limine-booted, higher-half kernel that reaches 64-bit long mode.
2. To implement low-level device drivers: a linear-framebuffer text console (bitmap font, scrolling), a 16550 UART serial driver, and a polled PS/2 keyboard.
3. To provide a small freestanding C runtime (memcpy/memset/printf) with no libc, and an interactive in-kernel command shell.
4. To design a clean, reproducible build (clang + GNU ld + a single Makefile + bootable ISO) that is testable in QEMU.
5. To build a companion graphical desktop that models OS concepts (windowing, apps, a sandboxed filesystem, networking) in user space.
6. To keep the desktop's network/Wi-Fi control sandboxed so the simulated OS can never disturb the host system.

3. Problem Statement

Operating-system internals — the boot handoff, the memory layout, device drivers and the shell — are often taught only in theory and feel inaccessible because real systems are enormous and their toolchains are intimidating. At the same time, graphical desktop environments hide how an OS presents processes, files and devices to the user. There is a need for a small, readable kernel that a student can build and boot in minutes, paired with an approachable graphical environment that makes

OS concepts (windows, a shell, a sandboxed filesystem, live network state) visible and interactive without ever putting the host machine at risk.

4. Related Work

1. Educational kernels such as MIT xv6 and the OSDev “Bare Bones” tutorials teach kernel fundamentals, but they often target older toolchains and stop short of a usable interactive shell.
2. *Writing an OS in Rust* (P. Oppermann) is an excellent guided kernel, but it is Rust-specific; NovaOS uses the more widely taught C path.
3. The **Limine** boot protocol hands the kernel a ready framebuffer and a higher-half mapping, removing much of the boilerplate that GRUB/Multiboot require.
4. Web-desktop projects (e.g. daedalOS) simulate an OS UI in a browser; NovaOS Desktop is instead a native PyQt5 application tied to a real kernel project.
5. System tools such as *htop* or GNOME System Monitor expose host state directly; NovaOS Desktop deliberately sandboxes its controls so it never changes the host configuration.

5. Scope

NovaOS targets the **x86_64** architecture only. The kernel boots through Limine on both BIOS and UEFI and currently provides framebuffer + serial output and an interactive shell; interrupts, virtual memory and multitasking are planned milestones rather than present features. NovaOS Desktop runs on a standard Linux desktop with Python 3 and a Qt binding (PyQt5, or PySide6 through a compatibility shim). Its file operations are confined to a sandboxed virtual drive, and its Wi-Fi manager is virtual: it reads nearby networks read-only but never alters the host's real connection.

6. Methodology

Kernel

1. Architecture: x86_64, loaded in the higher half at 0xffffffff80000000.
2. Bootloader: Limine, packaged as a hybrid BIOS/UEFI ISO (xorriso).
3. Language: freestanding C with minimal inline assembly.
4. Toolchain: clang (cross-compile via a bare-metal target triple), GNU ld, a single Makefile.
5. Drivers: linear-framebuffer console (8×8 font), 16550 UART serial, polled PS/2 keyboard.
6. Runtime: freestanding string/printf helpers and an interactive shell.
7. Testing: QEMU (qemu-system-x86_64).

NovaOS Desktop (companion application)

1. Language / UI: Python 3 with PyQt5 (PySide6 fallback via a binding shim).
2. Windowing: an MDI workspace (draggable/resizable windows), taskbar and start menu, with a live clock and a Wi-Fi indicator.
3. Applications: Terminal, Files, Web Browser, Network/Wi-Fi, Text Editor, Calculator, Settings and About.
4. Filesystem: a sandboxed virtual drive (~/.novaos_desktop/drive).
5. Browser: a lite engine (requests/urllib + QTextBrowser) with automatic upgrade to Qt WebEngine (Chromium) when it is installed.
6. Networking: read-only nmap scanning plus a virtual, sandboxed in-app Wi-Fi state.
7. Engineering: Git/GitHub, with feature work on a dev branch merged into main via pull request.

7. Design Principles

1. Small, single-purpose modules, each with a clear header; only one file ever touches the boot protocol, so the rest of the kernel is boot-agnostic.
2. Freestanding and dependency-light: the kernel ships only the libc routines it actually needs, and the desktop has zero image assets (its icons are painted at runtime).
3. A fast, readable build: using clang as the cross-compiler avoids bootstrapping a GCC toolchain, and the whole kernel builds from one Makefile.
4. Responsiveness and safety in the desktop: network and page I/O run on background threads, and the Wi-Fi manager is sandboxed so it can never change the host's real networking.

8. Limitations

1. The kernel is x86_64-only and currently has no interrupts, paging/virtual memory, multitasking or persistent filesystem (all planned milestones).
2. The desktop Browser's lite engine renders a simplified, JavaScript-free view; full CSS/JS pages require installing Qt WebEngine, after which the Browser switches to Chromium automatically.
3. NovaOS Desktop is a *simulation* of an OS desktop, not a real operating system, and depends on a host Linux machine with Qt.

9. Future Plans

For the kernel: a GDT/IDT with interrupt and exception handling, a PIT/APIC timer, a physical and virtual memory manager, then preemptive multitasking and a simple filesystem. For NovaOS Desktop: browser tabs and bookmarks, persistent settings, and additional applications such as an image viewer and a process/system monitor.

10. Conclusion

We can conclude that the project results in a working bare-metal x86_64 kernel that boots through Limine into 64-bit mode, prints to a framebuffer console and a serial port, and accepts commands through an interactive shell — verified end-to-end in QEMU. Its companion, NovaOS Desktop, provides an approachable graphical environment that models operating-system concepts — windowing, a shell, a sandboxed filesystem and networking — in user space, while a sandboxed Wi-Fi manager keeps the host system untouched. Together they make core OS ideas, from the boot handoff to a usable user environment, concrete, buildable and safe to explore.

11. References

- [1] Limine bootloader — <https://github.com/limine-bootloader/limine>
- [2] OSDev Wiki, Limine Bare Bones — https://wiki.osdev.org/Limine_Bare_Bones
- [3] P. Oppermann, Writing an OS in Rust — <https://os.phil-opp.com/>
- [4] MIT xv6 — <https://pdos.csail.mit.edu/6.828/>
- [5] Qt for Python (PyQt5 / PySide6) — <https://doc.qt.io/qtforpython/>
- [6] QEMU — <https://www.qemu.org/>
- [7] font8x8, public-domain bitmap font — <https://github.com/dhepper/font8x8>
- [8] Project repository — <https://github.com/rajesh-1920/novaos>